

Problem 1: Self assessment.

You treated every element up to y on level i as if FIND would have to touch it, summing $\sum_{j < r} P[\text{element } j \in \text{level } i] = (r - 1)p^i$, which ignores the fact that the search “skips over” long runs of nodes at level i whenever it descends from level- $i + 1$. That overcounts probes and gives a bound that vanishes as $p \rightarrow 0$, even though making p tiny actually turns the skip-list into a near-linear list and increases the expected work. The correct analysis tracks the gap back to the last level- $i + 1$ node and yields a geometric series summing to $(1 - p)/p$, which blows up as $p \rightarrow 0$.

Problem 2: Short-answer potpourri.

(a)

State, with justification, all the elements of $\mathbb{Z}_{11}^*$ that are generators of $\mathbb{Z}_{11}^*$.

Proof.

Since 11 is prime this implies that $\mathbb{Z}_{11}^*$ is a cyclic group of order 10, so elements will have order 1, 2, 5, 10, so we just have to check powers of 1, 2, 5 to see if they generate the group.

$2^2 = 4, 2^5 = 10$, so 2 is a generator.

$3^2 = 9, 3^5 = 1$

$4^2 = 5, 4^5 = 1$

$5^2 = 3, 5^5 = 1$

$6^2 = 3, 6^5 = 10$, so 6 is a generator.

$7^2 = 5, 7^5 = 10$, so 7 is a generator.

$8^2 = 9, 8^5 = 10$, so 8 is a generator.

$9^2 = 1, 9^5 = 1$

$10^2 = 1$

(b)

In the mystical realm of Numeria, each year consists of exactly 55 days, numbered from 0 to 54. One year, on the 0th day, the sages initiate a prophecy that the legendary Phoenix will return exactly 3^{281} days later.

Determine the day of the year when this will occur. Do not use any electronic computations, use as little by-hand calculation as you can, and show your work.

Proof.

$55 = 11 * 5$, and since $281 = 1 + (7)(10)(4)$ by Euler's Theorem we get that $3^{281} = 3$

(c)

What day of the week will it be 376^{280376} days after the Wednesday this homework is due? Show your work, and do not use a calculator.

Proof.

$376 = 5 \pmod{7}$. By Fermat's Little Theorem since 5, 7 are coprime this implies that $5^6 = 1$.

Thus, consider $280376 \pmod{6}$. Since it is even we know it is 0, 2, 4, 6.

So consider $280376 \pmod{3}$. Since the sum of the digits is 26 and $26 = 2 \pmod{3}$. This implies that $280376 = 2 \pmod{3}$.

Thus, $280376 = 2 \pmod{6}$.

Therefore, $5^{280376} = 5^2 = 4 \pmod{7}$. Thus, the day would be sunday. ■

Problem 3:

Suppose Alice uses $\text{KeyGen}()$ to produce (n, e, d) and publicly announces (n, e) , while Bob uses $\text{KeyGen}()$ to produce (n', e', d') and publicly announces (n', e') . Explain how to *efficiently* decode *any* encrypted message that you intercept, which was encrypted with (n, e) . You may assume $n \neq n'$.

Proof.

Because SecureCo reuses the same 512-bit prime p in every KeyGen run, any two different public moduli $n = pq$ and $n' = p q'$ necessarily share p , so an attacker can recover p in polynomial time by computing $\gcd(n, n')$. Once p is known, one divides n by p to obtain q , computes $\varphi(n) = (p - 1)(q - 1)$, and then uses the extended Euclidean algorithm to find the private exponent $d \equiv e^{-1} \pmod{\varphi(n)}$. With (n, d) thus reconstructed, any text C encrypted under Alice's public key can be decrypted by computing $C^d \bmod n$, breaking the encryption efficiently and completely. ■

Problem 4: Beyond eavesdropping.

In this question you will show that the “textbook” RSA encryption scheme described in class can be insecure under certain kinds of attacks that go beyond eavesdropping.

Recall that under “textbook” RSA, to encrypt a message m to Bob, whose public key is (n, e) , Alice sends $c = m^e \bmod n$. Suppose Malcolm *intercepts* c , so that Bob does not receive it. Instead, Malcolm chooses a suitable integer $r \not\equiv 1 \pmod{n}$, and sends $c' = c \cdot r^e \bmod n$ to Bob.

(a)

Derive the value m' that Bob obtains upon decrypting c' .

Proof.

Let $d \equiv e^{-1} \pmod{\varphi(n)}$ be Bob’s private key.

When Bob decrypts he will get $m' = (c')^d \bmod n = (c \cdot r^e)^d \bmod n = (mr)^{ed} = mr \bmod n$

(b)

Suppose that Bob reveals the value m' (from part (a)) to Malcolm—e.g., Bob might interpret it as “junk,” and reveal it in an error message. State what condition r should satisfy so that Malcolm can efficiently compute the original message m from m' , and explain how to do so.

Proof.

Pick an r s.t. $\gcd(r, n) = 1$. This implies that r has a multiplicative inverse in $\mathbb{Z}_n^*$, so once Bob gives $m' = m \cdot r \pmod{n}$. Malcolm only has to compute $m' * r^{-1} = m \pmod{n}$.

Problem 5: Diffie–Hellman as public-key encryption.

In class we said that Diffie–Hellman is an interactive protocol for two parties to establish a shared secret over a public channel, but it was not until RSA that a full public-key encryption scheme was known. However, it turns out that, by viewing the Diffie–Hellman protocol in the right way, we can actually use it for public-key encryption! (This perspective was discovered several years after RSA was proposed, and is widely used in practice now.)

Let p be a huge prime and g be a generator of $\mathbb{Z}_p^*$, both public. To generate a public key, Alice simply performs her “side” of the DH protocol: she chooses a secret exponent $a \in \mathbb{Z}_p^* = \{1, \dots, p-1\}$ uniformly at random, and publishes $x = g^a \pmod p$ as her public key.

(a)

Suppose that somebody (Bob, or Charlie, or Dan, etc.) wants to send Alice a confidential message m using her public key (and the other public data). Describe what the sender should do to accomplish this, and how Alice can recover the intended message from what is sent to her. (The algorithms run by Alice and the sender should be efficient.) Also give some brief intuition for why the method is plausibly secure against an eavesdropper. (You may assume that the DH protocol itself is secure, and that the sender has Alice’s genuine public key.)

Proof.

Alice’s public key is $x = g^a \pmod p$. To send her a message $m \in \mathbb{Z}_p^*$, Bob picks a random b , computes the key $c_1 = g^b \pmod p$ and the shared secret $k = x^b = g^{ab} \pmod p$, then masks the message as $c_2 = m \cdot k \pmod p$. He sends the pair (c_1, c_2) ; Alice recovers $k = (c_1)^a = g^{ab}$ and then retrieves

$$m = c_2 \cdot k^{-1} \pmod p.$$

An eavesdropper only sees $g, p, x = g^a, c_1 = g^b, c_2 = m g^{ab}$, and so would need to compute g^{ab} from g^a and g^b is the Diffie–Hellman problem. If the problem is hard then, the mask g^{ab} can’t be stripped off, and m stays confidential. ■

(b)

In class we mentioned a security problem with the oversimplified RSA encryption scheme, as we presented it. That is, the algorithm is *deterministic*, so encrypting the same message (under the same public key) always yields the same ciphertext. This allows an eavesdropper to detect repeated messages, or even do frequency analysis if the set of actually encrypted messages is not very large.

Explain why the DH-style encryption scheme from the previous part does *not* suffer from this specific problem.

Proof.

The Diffie-Hellman-style scheme avoids deterministic outputs by introducing fresh randomness into every encryption. Instead of computing a single fixed exponentiation of the message, the sender generates a new random exponent a each time, producing a randomized component $A = g^a$ and a shared secret $S = h^a = g^{ab}$. The ciphertext $(A, m \cdot S)$ therefore changes completely on each encryption because both A and the mask S depend on the new random a .

As a result, no two encryptions of the same plaintext under the same public key ever look identical, and an eavesdropper cannot spot repeats or do frequency analysis. This built-in randomness makes the scheme probabilistic (and semantically secure under DDH), in stark contrast to textbook RSA's deterministic nature. ■

Problem 6: zero-knowledge graph coloring.

Consider the following zero-knowledge proof that a given graph $G = (V, E)$ is 3-colorable.

- The prover (Merlin) starts with some fixed 3-coloring of G , and *recolors* it by randomly ‘shuffling’ the colors R,W,B. (For example, for the shuffling $R,W,B \rightarrow B,W,R$, the prover would recolor every R vertex as B, every W vertex as W, and every B vertex as R.) The prover then conceals each vertex with a removable opaque cover, and shows the graph with its concealed vertices to the verifier (Arthur).
- The verifier checks that the covered graph is indeed G (has the same vertices and edges), and then ‘challenges’ the prover by choosing an edge $(u, v) \in E$ uniformly at random.
- The prover uncovers just the vertices u and v , revealing their assigned colors.
- If u and v have *different* (valid) colors, the verifier accepts; otherwise, it rejects.

This basic iteration can be repeated multiple times (reshuffling the colors each time) to improve the soundness, but below we consider only a single iteration.

(a)

Suppose that G is *not* 3-colorable, but the prover tries to ‘fool’ the verifier into accepting. Give, with brief justification, the tightest possible upper bound on the probability (over the verifier’s random choice of challenge) that the prover can succeed.

Proof.

Let G be a graph that is not 3-colorable that the prover is try to cheat with. Let I be the number of incorrect edges meaning with two vertices of the same color.

The probability that the cheating prover’s succeeds on one iteration is the probability that the verifier does not pick an incorrect edge. Therefore,

$$1 - \frac{I}{|E|}$$

Since at least one edge must be incorrect this implies that

$$1 - \frac{I}{|E|} \leq 1 - \frac{1}{|E|}$$

(b)

Give a brief justification (1–2 sentences) why the protocol is zero knowledge.

Proof.

The only thing Arthur ever sees is two random, different colors on the endpoints of a single edge which he could himself simulate by just picking any two distinct colors at random. Thus, there is a simulator that produces exactly the same view as the real protocol without knowing any 3-coloring of G , so no additional information is leaked.

Problem 7: PTMR throwback: Final resolution.

In a large software project, each module depends on others. These dependencies are modeled as a directed graph $G = (V, E)$, where each vertex represents a module and a directed edge (u, v) means that module u *depends on* module v .

However, if there is a cycle in this graph, the system cannot resolve the dependencies, potentially causing infinite loops, circular imports, or build failures.

To restore a valid build order, we may need to remove or refactor certain modules to eliminate all cycles.

Formally, we define the language as:

$$\text{RESOLUTION} = \left\{ (G, k) : G = (V, E) \text{ and there exists a subset } V' \subseteq V \text{ of } \right. \\ \left. \text{(at most) } k \text{ vertices whose removal makes } G \text{ acyclic} \right\}.$$

Show $\text{VERTEX-COVER} \leq_p \text{RESOLUTION}$. Recall that

$$\text{VERTEX-COVER} = \{(G, k) : G \text{ has a vertex cover of size (at most) } k\}.$$

Since VERTEX-COVER is NP-hard, we conclude that RESOLUTION is also NP-hard.

Note: The graph in a VERTEX-COVER instance is an undirected graph, whereas the graph in a RESOLUTION instance is a directed graph.

Proof.

Let $f : \Sigma^* \rightarrow \Sigma^*$ s.t. $f(G = (V, E), k) = (G' = (V, E'), k)$ s.t. $E' = \{(u, v), (v, u) \mid \{u, v\} \in E\}$
Making G' takes $O(|G|)$ time thus polynomial.

Assume $(G, k) \in \text{VERTEX-COVER}$. Thus, there exists a $C \subset V$ s.t. C is a vertex cover of size $\leq k$. Consider $f(G, k) = (G', k)$.

Since C is a vertex cover in G and the only edges we add are between connected edges in G this implies that C is also a vertex cover in G' . Thus, let S be a directed cycle in G' . Consider $G' \setminus C$. Since C is also a vertex cover in G' this implies that at least one vertex of an edge in E' is in C . Thus, this implies that $S \cap C \neq \emptyset$. Therefore, when we remove C this implies that it will break S into two parts, and it will no longer be a cycle. Thus, $(G', k) \in \text{RESOLUTION}$.

Assume $f(G = (V, E), k) = (G' = (V, E'), k) \in \text{RESOLUTION}$. Let $C \subset V$ s.t. $G' \setminus C$ is acyclic s.t. $|C| \leq k$.

Assume C is not a vertex cover of G . This implies there exists an edge $u, v \in E$ s.t. $u, v \notin C$. This implies that $(u, v), (v, u) \in G' \setminus S$ which is a contradiction since this is a

two cycle. Thus, C is a vertex cover of G .

Thus, $(G, k) \in \text{VERTEX-COVER}$.

Therefore, $\text{VERTEX-COVER} \leq_p \text{RESOLUTION}$ ■

Problem 8:

The evil wizard Koshchei the Deathless has hidden his soul inside one of a dozen identical-looking eggs in a standard egg carton. Koshchei claims that he can sense which egg contains his soul, but he will destroy you if you try to crack any of the eggs and look inside.

Design, with justification, a zero-knowledge protocol by which Koshchei can convince you with 99.9% confidence that he is able to detect which egg contains his soul. (In other words, if Koshchei does *not* have this power, the probability that he convinces you in the protocol should be at most 0.1%.) You have access to a place where Koshchei cannot spy on you.

Proof.

First put the carton of eggs into a box that is identical on all sides so that the carton is perpendicular to one of the sides. Then place the box on a rotating tray. Spin the box randomly, so you know do not know which way the carton is facing. Then sit at a table across from Koshchei. Tell him to sense where the egg is but to not say anything. Then go to a secret room and randomly decide to rotate the tray 180° or to do nothing. Then bring the tray back and ask Koshchei if you rotated the tray. Repeat this 10 times.

If he can sense his special egg then he will be able to always tell if you rotated the tray or not.

If he does not know then there is a $(1/2)^{10} \approx 0.09\% < 0.1\%$ chance of guessing them all right. Thus, he can fool you with probability at most $(1/2)^{10}\%$.

This is Zero Knowledge because an honest execution can be simulated by the verifier alone, using only our random choice to rotate the box or not, without talking to Koshchei.

If he can tell which egg is correct then he will always pick correct causing us to trust him, and we always know whether we rotated or not, so we know what Koshchei will say in order to convince us. ■